



두비두팝 (React Native + Expo / Web) 보컬 커스터마이징 스튜디오 개발

1. 프로젝트 개요

- **프로젝트명:** 두비두팝 보컬 커스터마이징 스튜디오 개발
- **개발 기간:** 2026.06 ~ 2026.06
- **플랫폼/환경:** React Native + Expo / Web + TypeScript
- **담당 범위**
 - Figma-first 기반 스튜디오 기반 퍼블리싱(Home / Edit / Preview / Publish / Loading)
 - 제작(EditStage) 인터랙션: 스템 선택/해제, 라인 전체 선택/배정, 드래그&드롭, Undo·Redo
 - 프리뷰(PreviewStage) 재생 제어 / 검증 플로우 / 모달
 - 발행(PublishStage) / 로딩>LoadingStage) 안정화 및 픽셀 피델리티
 - 공통 인터랙티브 컴포넌트 표준화 + 전역 상태 구조 정리
 - mock 기반 시나리오 구현 + 스튜디오 V2 핵심 인터랙션
 - 모바일 최적화, 곡 데이터 연동, Vercel·EAS OTA 배포 파이프라인

2. 요구사항 및 나의 역할

요구사항 요약

- Figma 시안 기준으로 전체 사용자 여정을 빠르게 “사용 가능한” 형태로 세워야 했음
- 제작 단계 스템 조작은 실수가 결과물 품질로 직결되므로 안정적인 인터랙션 모델이 필요했음

- 프리뷰는 사용자가 결과물을 신뢰하는 구간이라 검증/재생 상태가 명확해야 했음
- 발행/로딩은 기능보다 안정성/신뢰가 중요했음

담당 역할

- **기반 퍼블리싱**: Figma 시안 기준 Home/Edit/Preview/Publish/Loading 화면 구조와 기본 플로우 구현
- **제작 인터랙션 구현**: stemSlots 모델링, 드래그&드롭 hover 판정, 라인 선택/배정 모드, Undo/Redo 2단계 히스토리
- **프리뷰 구현**: 재생/원곡 동시 재생 방지, 검증 상태 모델, 전문보기/결과 모달
- **발행/로딩 안정화**: 빌드/런타임 오류 제거, 발행 옵션 UI, 로딩 픽셀 피델리티
- **공통화/배포**: Interactive 컴포넌트 표준화, 모바일 최적화, Vercel·EAS 배포 파이프라인 구성

3. 기술 스택 및 아키텍처

항목	사용 기술 / 접근 방식
프레임워크	React Native, Expo, Web, TypeScript
상태/히스토리	stemSlots 모델링, Undo/Redo 2단계 스택, verifyState(idle → error → verified)
공통 컴포넌트	InteractiveIconButton / ListItem / UtilityButton, 상태 체계 (Default/Active/Pressed/Disable)
데이터/배포	mockOptions.ts, S3 오디오/artist rail, Vercel Production, EAS preview OTA

아키텍처 특징

- Figma-first + mock 기반으로 전체 플로우를 먼저 세우고, 화면/상태 뼈대는 유지한 채 데이터만 실 API로 교체 가능한 구조
- 반복 UI를 Interactive 컴포넌트로 표준화하고 상태 규칙 (Default/Active/Pressed/Disable)을 통합해 회귀 감소
- 검증/재생 상태를 명시적 상태 모델로 관리해 오작동·중복 실행 방지

4. 주요 기능

기능	설명
기반 퍼블리싱	Home/Edit/Preview/Publish/Loading 화면 구조 및 기본 플로우
제작(EditStage)	스텝 선택/해제, 라인 전체 선택/배정 모드, 드래그&드롭, Undo/Redo 2단계
프리뷰(PreviewStage)	재생/원곡 동시 재생 방지, 검증 플로우, 전문보기/결과 모달, 스낵바
발행(PublishStage)	공개 범위/소장 옵션 드롭다운, 팝업 상호 배타 토글, 약관 동의
로딩>LoadingStage)	실제 PNG 에셋 교체, 캔버스(852×348) 제약 내 레이아웃 안정화
공통 컴포넌트	Interactive 컴포넌트 표준화 및 상태 체계 통합
mock 시나리오	mockOptions.ts로 데이터 분리, 스튜디오 V2 핵심 인터랙션
모바일/배포	Android 가로·Safe Area 대응, 곡 데이터 연동, Vercel·EAS OTA

5. 주요 성과

- **Figma-first 기반 핵심 플로우를 mock 기반으로 빠르게 구축:** Home → 제작 → 프리뷰 → 발행 → 로딩 전체 여정을 연결하고 이후 실 API 연동 가능한 구조 마련
- **제작 단계 스텝 조작 UX를 안정적 인터랙션 모델로 구현:** stemSlots 모델링, 드래그&드롭(겹침 면적 기반 hover 판정), 선택/라인 배정 모드, Undo/Redo 2단계 히스토리로 조작 실수와 상태 혼선 감소
- **프리뷰 검증·재생 상태를 명확한 상태 모델로 구조화:** idle → error → verified 모델과 재생/원곡 동시 재생 방지, 재생 중 UI disable로 오작동·중복 실행 방지
- **발행/로딩 빌드·런타임 안정화:** ES2021 replaceAll typecheck 실패, React key 타입 이슈, 누락 import 크래시(검은 화면)를 제거하고 로딩 화면을 픽셀 단위로 안정화
- **공통 인터랙티브 컴포넌트 표준화 + 모바일 최적화·배포 연결:** 상태 규칙 통합으로 QA/유지보수 기반 마련, Android 가로/Safe Area 대응, 곡 데이터(TWICE TT 등) 연동, Vercel·EAS OTA 파이프라인 구성

6. 문제 해결 및 기술적 도전

✅ 문제 1: 실 API 이전에 전체 플로우를 빠르게 세워야 했던 문제

실 API 연동 전 단계에서 핵심 유저 플로우(진입 → 제작 → 프리뷰 → 발행 → 로딩)를 mock 기반으로 끊임 없이 구현하고, mockOptions.ts로 하드코딩 데이터를 분리해 App.tsx 복잡도를 줄였음. 이후 API가 붙을 때 화면/상태 뼈대는 유지하고 데이터만 교체하는 방식으로 속도를 확보.

✓ 문제 2: 복잡한 스템 조작 UX에서 조작 실수를 줄여야 했던 문제

스스템 조합 UI는 조작 실수가 결과물 품질로 직결됨. 아티스트 ↔ 스템 연결을 stemSlots로 모델링하고, 드래그 중 고스트 카드 겹침 면적 기반 hover 판정으로 드롭 일관성을 확보했으며, lineSelect/lineAssign 모드를 분리하고 Undo/Redo 2단계를 도입해 작업 속도와 오류율을 함께 개선했음.

✓ 문제 3: 사용자가 결과물을 신뢰할 수 있는 프리뷰 검증을 만들어야 했던 문제

idle → error → verified 검증 상태 모델을 도입해 “확인하기” 클릭 시 로딩 오버레이/에러 체크/스แน바/결과 모달까지 실제 플로우로 구성하고, verified 상태에서만 “발행하기”를 활성화했음. 재생/원곡 동시 재생을 방지하고 재생 중 일부 UI를 disable 처리해 오작동·중복 실행을 줄였음.

✓ 문제 4: 발행/로딩 구간의 빌드·런타임 안정성을 확보해야 했던 문제

ES2021 replaceAll 사용으로 인한 typecheck 실패를 정규식 replace로 교체하고, React key 타입 이슈와 누락 import로 인한 검은 화면 크래시를 파악·복구했음. 로딩 화면은 임시 도형을 실제 PNG 에셋으로 교체하고 캔버스(852×348) 제약 내 잘림/줄바꿈을 레이아웃으로 안정화했음.

✓ 문제 5: 화면이 늘어나도 일관성을 유지해야 했던 문제

GNB/리스트/유틸리티 버튼 등 반복 UI를 Interactive 컴포넌트로 표준화하고 Default/Active/Pressed/Disable 상태 규칙을 통합했음. 스크롤 시 모달 잘림은 Modal 기반 오버레이(뷰포트 고정)로, 롱프레스 배경 깜빡임은 제거해 “버튼마다 다른 규칙”으로 생기는 회귀를 줄였음.

7. 협업 및 커뮤니케이션

- Figma 노드별 contract/QA 문서를 기록해 디자인-개발 간 해석 차이를 줄임
- 커밋 단위 작업 기록(빌드/타입체크 통과, 변경 파일 요약)으로 진행 상황과 변경 범위를 투명하게 공유
- 공통 컴포넌트/상태 규칙을 표준화해 QA/리뷰 커뮤니케이션 비용을 낮춤

8. 개발 결과 및 회고



결과

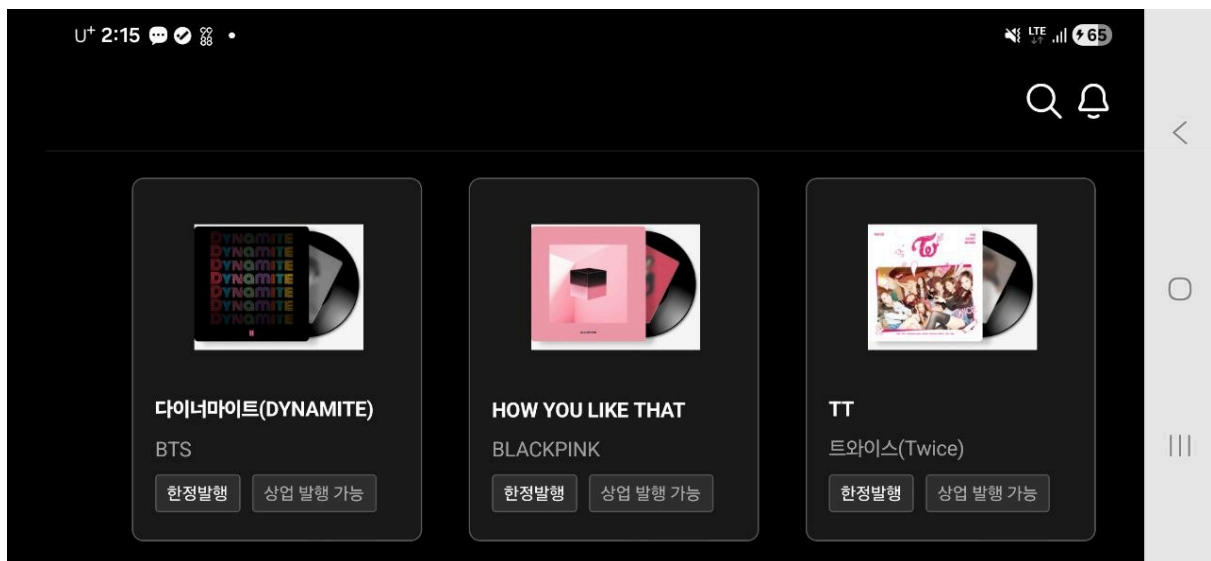
- 제작/프리뷰/발행/로딩 전 구간의 UI·상태·검증 플로우를 mock 기반으로 완성하고 스튜디오 V2 핵심 인터랙션까지 구현
- 모바일 최적화 및 Vercel·EAS OTA 배포 파이프라인까지 연결

🤔 회고

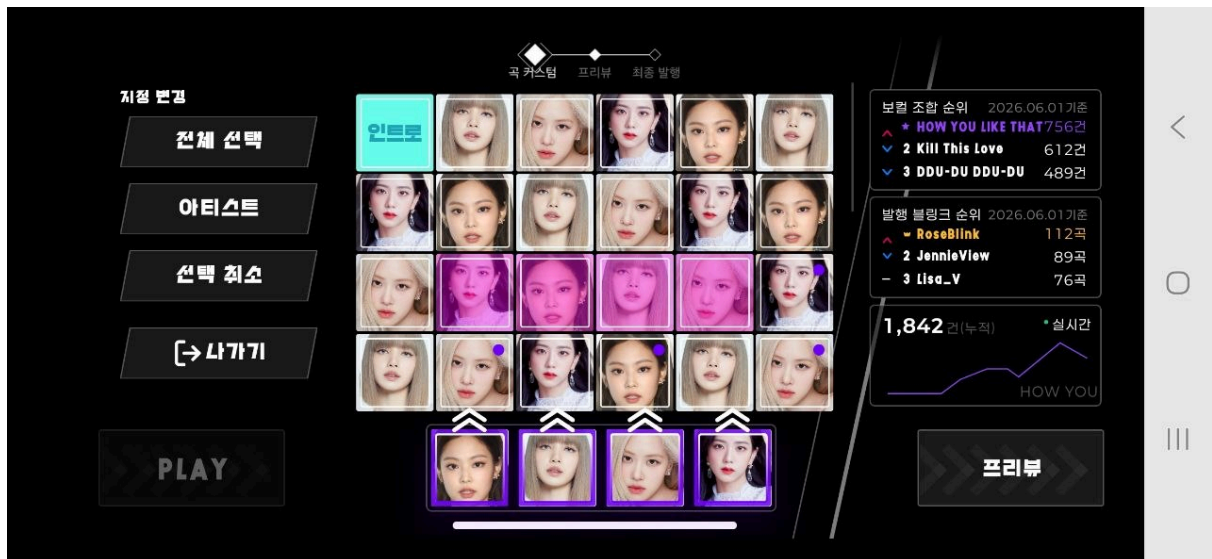
- 인터랙션이 많은 제작 도구는 기능 추가보다 선택/배정/되돌리기 같은 기본 인터랙션과 상태 규칙을 초기에 단단히 잡는 것이 이후 확장 속도와 품질을 좌우했음

9. 스크린샷

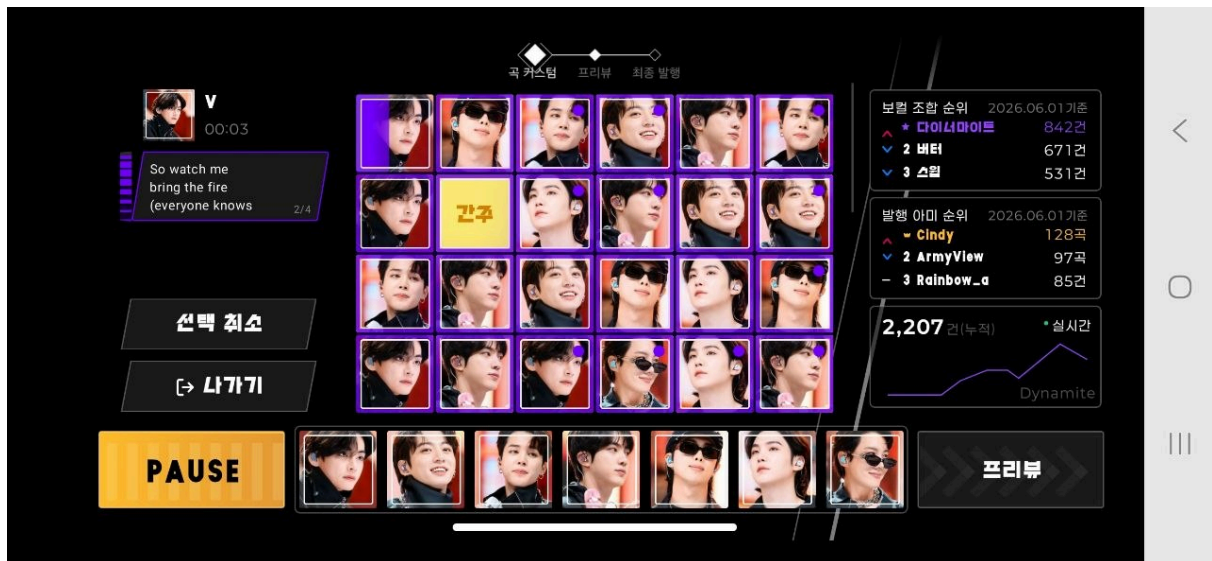
- 제작(EditStage) 화면 / 프리뷰 검증 모달 / 발행 옵션 팝업 / 로딩 화면 / 스튜디오 V2 등 주요 화면 캡처



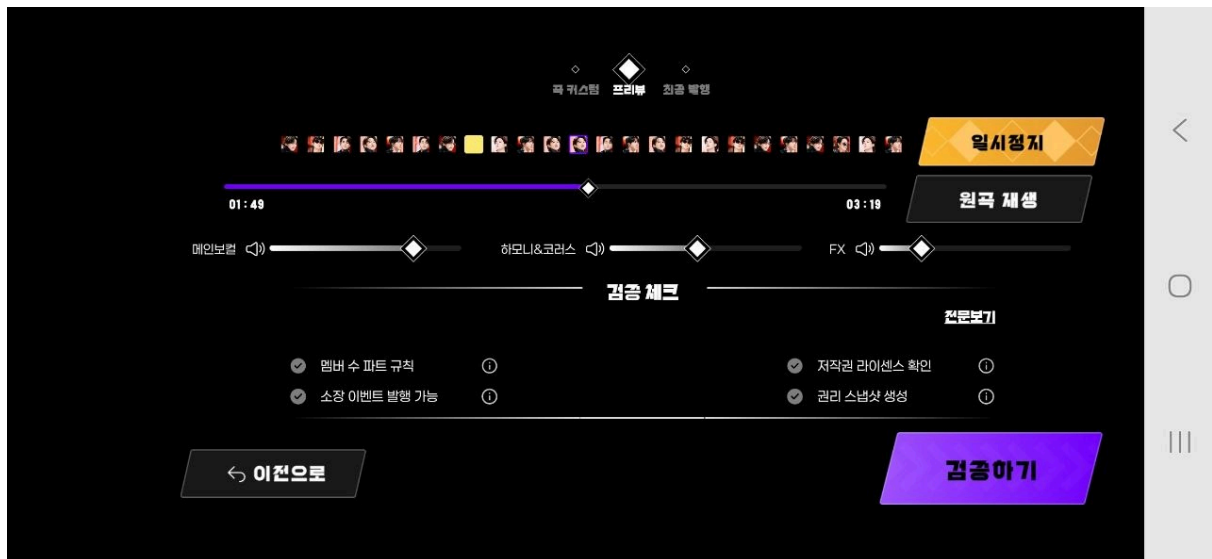
이미지 1. 리스트 화면



이미지 2. 제작 모드 화면



이미지 3. 재생 모드 화면



이미지 4. 프리뷰 검증 화면



이미지 5. 발행 화면